

UT 06.02 - El Lenguaje DQL. JOINS.



Autor: Carlos Moreno Martínez.
cmorenomartinez@educa.madrid.org



UT 06.02 - El Lenguaje DQL. JOINS.

1.- Introducción.

- El análisis y diseño que se realiza para modelizar un sistema mediante una base de datos gestionada mediante un SGBD. En este análisis se van a crear las tablas necesarias para contener la información.
- En muchos casos, va a ser necesario confeccionar informes con campos **residentes en distintas tablas mediante las sentencias multitabla o JOINS**.
- En este tipo de consultas se van a aprovechar las características de los SGBD tales como:
 - Las características de una clave primaria (obligatoriedad y unicidad).
 - Conservación de la integridad referencial, etc.
 - Casos que se tendrá que tener especial cuidado, sobre todo cuando aparecen valores NULL en campos determinados.
- Con el avance de la sociedad de la información, el tratamiento y el relacionado de cantidades masivas de datos procedentes de numerosas fuentes de datos constituye la base del llamado **Big Data**.

El estándar ANSI SQL.

- En Oracle existen dos formas de hacer una unión entre tablas:
- **Clásica**. Está disponible en todas las versiones, pero no cumple con los estándares ANSI.
- **Estándar**. Cumple con la regulación ANSI ISO SQL 99.

UT 06.02 - El Lenguaje DQL. JOINS.

2.- Composición interna.

2.1.- Producto Cartesiano. Definición.

- Consiste en una combinación de las filas de las dos tablas independientemente de si están relacionadas o no.
- Una vez combinadas, se pueden filtrar para conseguir las que cumplan determinada condición. También se pueden usar como base para sentencias mas sofisticadas tales como sentencias de agrupamiento o subconsultas.
- El formato de un producto cartesiano es el siguiente:

SELECT

campos

FROM tabla1 alias1, tabla2 alias2, ... ;

UT 06.01 - El Lenguaje DQL. Agrupamientos.

2.- Composición interna.

2.1.- Producto Cartesiano. Ejemplos.

```
-- MOSTRAR UN LISTADO CON LAS PAREJAS DE (MANAGER, CLERK) CUYA SUMA  
-- DE SUS SALARIOS SEA MENOR QUE 3,800.
```

```
SELECT  
    M.ENAME MANAGER, M.SAL SAL_MANAGER,  
    C.ENAME CLERK, C.SAL SAL_CLERK,  
    M.SAL + C.SAL TOTAL  
FROM EMP M, EMP C  
WHERE M.JOB = 'MANAGER' AND C.JOB = 'CLERK'  
      AND M.SAL + C.SAL < 3800;
```

MANAGER	SAL_MANAGER	CLERK	SAL_CLERK	TOTAL
JONES	2975	SMITH	800	3775
BLAKE	2850	SMITH	800	3650
CLARK	2450	SMITH	800	3250
CLARK	2450	ADAMS	1100	3550
CLARK	2450	JAMES	950	3400
CLARK	2450	MILLER	1300	3750

```
-- EN BASE A LOS RESULTADOS DE LA CONSULTA ANTERIOR MOSTRAR EL NUM DE CLERK  
-- POR CADA MANAGER QUE CUMPLEN ESA PREMISA.
```

```
SELECT  
    M.ENAME MANAGER, COUNT(*) NUM_CLERK  
FROM EMP M, EMP C  
WHERE M.JOB = 'MANAGER' AND C.JOB = 'CLERK' AND M.SAL + C.SAL < 3800  
GROUP BY M.ENAME;
```

MANAGER	NUM_CLERK
JONES	1
CLARK	4
BLAKE	1

UT 06.02 - El Lenguaje DQL. JOINS.

2.- Composición interna.

2.2.- CROSS JOIN.

- En **ANSI ISO SQL 99**, el producto cartesiano se define mediante una **CROSS JOIN** ó **unión cruzada**. Una CROSS JOIN produce el mismo efecto que en un producto cartesiano. Es decir, cada fila de la primera tabla se combina con cada una de las filas de la segunda tabla.
- El formato es el siguiente:

SELECT

campos

FROM tabla1 alias1 CROSS JOIN tabla2 alias2;

```
-- MOSTRAR UN LISTADO CON LAS PAREJAS DE (MANAGER, CLERK) CUYA SUMA  
-- DE SUS SALARIOS SEA MENOR QUE 3,800.
```

```
SELECT
```

```
    M.ENAME MANAGER, M.SAL SAL_MANAGER,  
    C.ENAME CLERK, C.SAL SAL_CLERK,  
    M.SAL + C.SAL TOTAL
```

```
FROM EMP M CROSS JOIN EMP C
```

```
WHERE M.JOB = 'MANAGER' AND C.JOB = 'CLERK'  
      AND M.SAL + C.SAL < 3800;
```

MANAGER	SAL_MANAGER	CLERK	SAL_CLERK	TOTAL
JONES	2975	SMITH	800	3775
BLAKE	2850	SMITH	800	3650
CLARK	2450	SMITH	800	3250
CLARK	2450	ADAMS	1100	3550
CLARK	2450	JAMES	950	3400
CLARK	2450	MILLER	1300	3750

UT 06.02 - El Lenguaje DQL. JOINS.

3.- Unión natural. NATURAL JOIN.

- La unión natural se basa en **relacionar tablas mediante campos en ambas tablas con el mismo nombre y características, seleccionando las columnas de ambas que tienen valores coincidentes.**
- El formato de la sentencia es el siguiente:

```
SELECT  
    campos  
FROM tabla1 alias1 NATURAL JOIN tabla2 alias2;
```

```
-- MOSTRAR UN INFORME CON EL NOMBRE DE LOS MANAGER Y LA LOCALIDAD  
-- DONDE TRABAJAN.  
SELECT  
    E.ENAME MANAGER,  
    D.LOC LOCALIDAD  
FROM EMP E NATURAL JOIN DEPT D  
WHERE E.JOB = 'MANAGER';
```

MANAGER	LOCALIDAD
CLARK	NEW YORK
JONES	DALLAS
BLAKE	CHICAGO

- Esta NATURAL JOIN es posible realizarla ya tanto la tabla EMP como la tabla DEPT tienen un campo DEPTNO que es NUMBER(2).

UT 06.02 - El Lenguaje DQL. JOINS.

4.- Uniones de Tablas. JOIN.

4.1.- Clausula USING.

- Permite definir el campo de unión entre las tablas. El campo puede tener distinto tipo de dato en cada una de las tablas.
- La sintaxis de la sentencia es la siguiente:

```
SELECT
    campos
FROM tabla1 JOIN tabla2 USING (campo);
```

- Las tablas no deben tener alias. Si en el resultado se quiere incluir campos de ambas tablas que tengan el mismo nombre se producirá un error.

```
-- MOSTRAR UN INFORME CON EL NOMBRE DE LOS MANAGER Y LA LOCALIDAD
-- DONDE TRABAJAN.
SELECT
    ENAME MANAGER,
    LOC LOCALIDAD
FROM EMP JOIN DEPT USING (DEPTNO)
WHERE JOB = 'MANAGER';
```

MANAGER	LOCALIDAD
-----	-----
CLARK	NEW YORK
JONES	DALLAS
BLAKE	CHICAGO

UT 06.02 - El Lenguaje DQL. JOINS.

4.- Uniones de Tablas. JOIN.

4.2.- Clausula ON. Unión con condición de igualdad.

- No siempre la unión se realiza mediante campos con el mismo nombre o con una condición de igualdad. Para estos casos, se tiene que usar la clausula ON.
- La sintaxis en este caso es la siguiente.

SELECT
 campos
FROM tabla1 alias1 **JOIN** tabla2 alias2 **ON** (condición);

- Veamos un ejemplo en el cual se usa una **condición de igualdad** para realizar la operación de JOIN.

```
-- MOSTRAR NOMBRE DE EMPLEADO Y NOMBRE DEL DPTO DONDE TRABAJA PARA
-- TODOS LOS EMPLEADOS QUE GANAN MAS DE 2,500.
SELECT
    E.ENAME MANAGER,
    D.LOC LOCALIDAD
FROM EMP E JOIN DEPT D ON (E.DEPTNO = D.DEPTNO)
WHERE E.SAL > 2500;
```

MANAGER	LOCALIDAD
-----	-----
JONES	DALLAS
BLAKE	CHICAGO
SCOTT	DALLAS
KING	NEW YORK
FORD	DALLAS

UT 06.02 - El Lenguaje DQL. JOINS.

4.- Uniones de Tablas. JOIN.

4.2.- Clausula ON. Unión con condición distinta a la igualdad.

- En este ejemplo se unen la tabla EMP y la tabla SALGRADE mediante **una condición que no es de igualdad**.

```
-- MOSTRAR NOMBRE DE EMPLEADO Y CATEGORIA SALARIAL DE LOS  
-- EMPLEADOS ASIGNADOS CON UN SALARIO SUPERIOR A 2,500.  
SELECT  
    E.ENAME,  
    S.GRADE  
FROM EMP E JOIN SALGRADE S ON (E.SAL BETWEEN S.LOSAL AND S.HISAL)  
WHERE E.SAL > 2500;
```

ENAME	GRADE
JONES	4
BLAKE	4
SCOTT	4
FORD	4
KING	5

UT 06.02 - El Lenguaje DQL. JOINS.

4.- Uniones de Tablas. JOIN.

4.2.- Clausula ON. Unión de mas de dos tablas simultáneamente.

- Es posible realizar la unión de **todas las tablas que sea necesario**.
- Para ello, se realiza el enlace de las dos primeras, el resultado se une con la tercera y así sucesivamente.

```
SELECT
    campos
FROM tabla1 alias1 JOIN tabla2 alias2 ON (condición1)
JOIN tabla3 alias3 ON (condición2)
.....;
```

```
-- MOSTRAR NOMBRE DE EMPLEADO, NOMBRE DE DPTO Y CATEGORIA SALARIAL
-- DEL EMPLEADO DE TODOS LOS EMPLEADOS QUE SEAN CLERK.
SELECT
    E.ENAME,
    D.DNAME,
    S.GRADE
FROM EMP E JOIN DEPT D ON (E.DEPTNO = D.DEPTNO)
JOIN SALGRADE S ON (E.SAL BETWEEN S.LOSAL AND S.HISAL)
WHERE E.JOB = 'CLERK';
```

ENAME	DNAME	GRADE
MILLER	ACCOUNTING	2
ADAMS	RESEARCH	1
JAMES	SALES	1
SMITH	RESEARCH	1

UT 06.02 - El Lenguaje DQL. JOINS.

5.- Uniones externas. Definición del problema.

- En muchas ocasiones no solo es necesario que aparezcan los datos que cumplen las condiciones de unión sino que deben aparecer algunas de las filas que no aparecen.
- Si realizamos un informe del nombre del departamento y el número de empleados que tiene muestra los siguiente:

```
SELECT * FROM DEPT;
```

DEPTNO	DNAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

```
SELECT
  D.DNAME,
  COUNT(*) "NUM EMP"
FROM EMP E JOIN dept D ON (E.DEPTNO = D.DEPTNO)
GROUP BY D.dname;
```

DNAME	NUM EMP
ACCOUNTING	3
RESEARCH	5
SALES	6

UT 06.02 - El Lenguaje DQL. JOINS.

5.- Uniones externas.

5.1.- Uniones por la izquierda. LEFT OUTER JOIN (I).

- En una union **LEFT OUTER JOIN** se muestran los registros de la tabla que aparece a la izquierda de la unión (antes de las palabras LEFT OUTER JOIN) que no tienen correspondencia en la otra tabla.
- La sintaxis es la siguiente:

SELECT

campos

FROM tabla1 alias1 LEFT OUTER JOIN tabla2 alias2 ON (condición);

```
SELECT E.ENAME, D.DNAME  
FROM DEPT D LEFT OUTER JOIN EMP E ON (D.DEPTNO = E.DEPTNO);
```

ENAME	DNAME
CLARK	ACCOUNTING
KING	ACCOUNTING
MILLER	ACCOUNTING
JONES	RESEARCH
FORD	RESEARCH
ADAMS	RESEARCH
SMITH	RESEARCH
SCOTT	RESEARCH
WARD	SALES
TURNER	SALES
ALLEN	SALES
JAMES	SALES
BLAKE	SALES
MARTIN	SALES
	OPERATIONS

UT 06.02 - El Lenguaje DQL. JOINS.

5.- Uniones externas.

5.1.- Uniones por la izquierda. LEFT OUTER JOIN (II).

- En la consulta anterior hay un total de quince registros:
 - Los primeros catorce registros son los catorce empleados que hay en la tabla EMP.
 - La última fila tiene el campo nombre de empleado (E.ename) a NULL y el nombre del departamento (D.DNAME) con el valor 'OPERATIONS'. **Esto indica que dicho departamento de la tabla DEPT (parte izquierda del JOIN) no tiene una correspondencia en la tabla EMP (parte derecha del JOIN).**
- Por tanto, para listar el nombre del empleado y el número de personas que trabajan en él incluyendo al departamento OPERATIONS tendríamos que contar el número de registros que tienen el campo E.ENAME (nombre del empleado) con un valor distinto de NULL.

```
SELECT
    D.DNAME,
    COUNT(E.ENAME) NUM_EMP
FROM DEPT D LEFT JOIN EMP E ON (D.DEPTNO = E.DEPTNO)
GROUP BY D.DNAME;
```

DNAME	NUM_EMP
ACCOUNTING	3
OPERATIONS	0
RESEARCH	5
SALES	6

UT 06.02 - El Lenguaje DQL. JOINS.

5.- Uniones externas.

5.2.- Uniones por la derecha. RIGHT OUTER JOIN (I).

- En este caso, se buscan que elementos de la tabla que está a la derecha de la unión la tabla de la parte izquierda. La sintaxis es la siguiente:

SELECT

campos

FROM tabla1 alias1 RIGHT OUTER JOIN tabla2 alias2 ON (condición);

- Veamos un ejemplo: **Indicar que empleados no tienen subordinados a su cargo.** Para ello, se buscan los empleados no tienen superior; es decir se busca que elementos tienen un valor NULL en la columna E.ename.

```
SELECT
```

```
    E.ENAME EMPLEADO,
```

```
    S.ENAME SUPERIOR
```

```
FROM EMP E RIGHT JOIN EMP S ON (E.MGR = S.EMPNO);
```

```
EMPLEADO
```

```
SUPERIOR
```

```
-----
```

```
SMITH
```

```
FORD
```

```
ALLEN
```

```
BLAKE
```

```
WARD
```

```
BLAKE
```

```
JONES
```

```
KING
```

```
MARTIN
```

```
BLAKE
```

```
BLAKE
```

```
KING
```

```
CLARK
```

```
KING
```

```
SCOTT
```

```
JONES
```

```
TURNER
```

```
BLAKE
```

```
ADAMS
```

```
SCOTT
```

```
JAMES
```

```
BLAKE
```

```
FORD
```

```
JONES
```

```
EMPLEADO
```

```
SUPERIOR
```

```
-----
```

```
MILLER
```

```
CLARK
```

```
TURNER
```

```
WARD
```

```
MARTIN
```

```
ALLEN
```

```
MILLER
```

```
SMITH
```

```
ADAMS
```

```
JAMES
```


UT 06.02 - El Lenguaje DQL. JOINS.

5.- Uniones externas.

5.2.- Uniones por la derecha. RIGHT OUTER JOIN (II).

El resultado de la consulta es por tanto:

```
SELECT
  S.ENAME
FROM EMP E RIGHT JOIN EMP S ON (E.MGR = S.EMPNO)
WHERE E.ENAME IS NULL;
```

ENAME

TURNER

WARD

MARTIN

ALLEN

MILLER

SMITH

ADAMS

JAMES

UT 06.02 - El Lenguaje DQL. JOINS.

5.- Uniones externas.

5.3.- Uniones por ambos lados. FULL OUTER JOIN. Definición.

- Puede haber casos en los cuales es necesario realizar las dos uniones anteriores de forma simultanea. Para este caso está la **FULL OUTER JOIN**. En este caso, aparecen los registros propios de la **unión interna**, la externa por la izquierda (**LEFT JOIN**) y la externa por la derecha (**RIGHT JOIN**).
- La sintaxis es la siguiente:

SELECT

campos

FROM tabla1 alias1 FULL OUTER JOIN tabla2 alias2 ON (condición);

- Muchas veces es muy útil realizar uniones de todas las tablas de una base de datos mediante FULL OUTER JOINS. Sabiendo interpretar el resultado de esta consulta (pueden llegar a obtenerse miles de registros) se facilitan muchos las consultas.

UT 06.02 - El Lenguaje DQL. JOINS.

5.- Uniones externas.

5.3.- Uniones por ambos lados. FULL OUTER JOIN. Ejemplo (I).

Veamos un ejemplo. Vamos a obtener la relación de empleados y subordinados completa: que empleado es subordinado de quien (verde), que empleado no es subordinado de nadie (azul) y que empleados no tienen subordinados (negro).

```
SELECT
    SUB.ENAME SUBORDINADO,
    SUP.ENAME SUPERIOR
FROM EMP SUB FULL OUTER JOIN EMP SUP ON (SUB.MGR = SUP.EMPNO);
```

SUBORDINAD	SUPERIOR	SUBORDINAD	SUPERIOR
-----	-----	-----	-----
	SMITH	BLAKE	KING
	ALLEN	JONES	KING
	WARD		TURNER
FORD	JONES		ADAMS
SCOTT	JONES		JAMES
	MARTIN	SMITH	FORD
JAMES	BLAKE		MILLER
TURNER	BLAKE	KING	
MARTIN	BLAKE		
WARD	BLAKE		
ALLEN	BLAKE		
MILLER	CLARK		
ADAMS	SCOTT		
CLARK	KING		

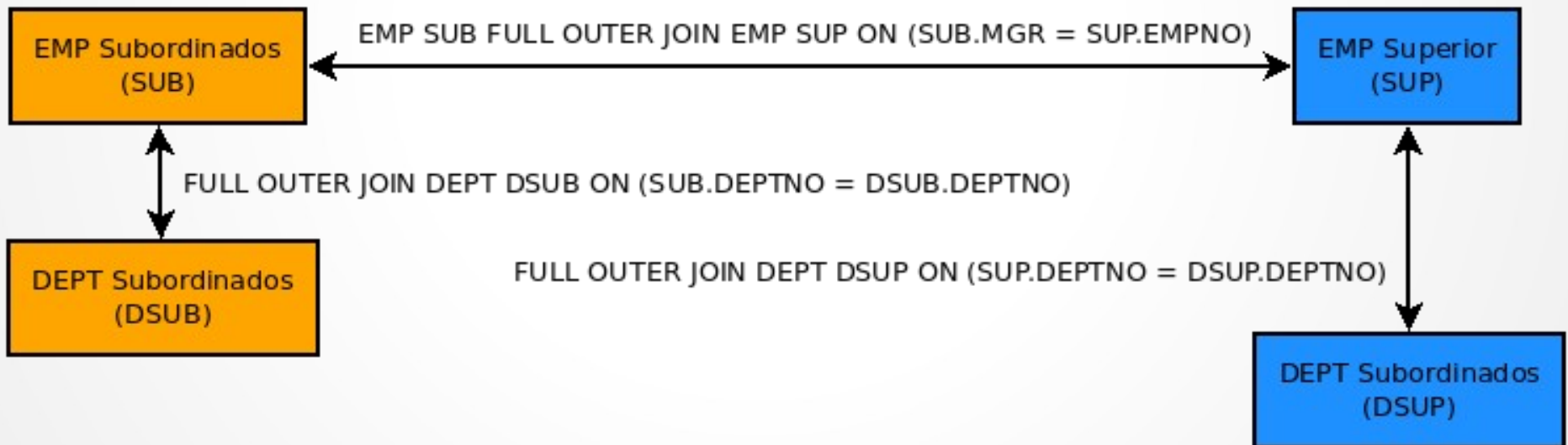
UT 06.02 - El Lenguaje DQL. JOINS.

5.- Uniones externas.

5.3.- Uniones por ambos lados. FULL OUTER JOIN. Ejemplo (II).

En este ejemplo unimos las tablas emp y dept para obtener cuatro campos: nombre del empleado, nombre del departamento al que pertenece el empleado, nombre del superior, nombre del departamento al que pertenece el superior.

```
SELECT
    SUB.ENAME SUBORDINADO,
    DSUB.DNAME DPTOSUB,
    SUP.ENAME SUPERIOR,
    DSUP.DNAME DPTOSUP
FROM EMP SUB FULL OUTER JOIN EMP SUP ON (SUB.MGR = SUP.EMPNO)
FULL OUTER JOIN DEPT DSUB ON (SUB.DEPTNO = DSUB.DEPTNO)
FULL OUTER JOIN DEPT DSUP ON (SUP.DEPTNO = DSUP.DEPTNO);
```



UT 06.02 - El Lenguaje DQL. JOINs.

5.- Uniones externas.

5.3.- Uniones por ambos lados. FULL OUTER JOIN. Ejemplo (III).

```
SELECT
    SUB.ENAME SUBORD,
    DSUB.DNAME DPTOSUB,
    SUP.ENAME SUPERIOR,
    DSUP.DNAME DPTOSUP
FROM EMP SUB FULL OUTER JOIN EMP SUP ON (SUB.MGR = SUP.EMPNO)
FULL OUTER JOIN DEPT DSUB ON (SUB.DEPTNO = DSUB.DEPTNO)
FULL OUTER JOIN DEPT DSUP ON (SUP.DEPTNO = DSUP.DEPTNO);
```

SUBORD	DPTOSUB	SUPERIOR	DPTOSUP	SUBORD	DPTOSUB	SUPERIOR	DPTOSUP
-----	-----	-----	-----	-----	-----	-----	-----
		SMITH	RESEARCH			TURNER	SALES
		ALLEN	SALES			ADAMS	RESEARCH
		WARD	SALES			JAMES	SALES
FORD	RESEARCH	JONES	RESEARCH	SMITH	RESEARCH	FORD	RESEARCH
SCOTT	RESEARCH	JONES	RESEARCH			MILLER	ACCOUNTING
		MARTIN	SALES	KING	ACCOUNTING		
JAMES	SALES	BLAKE	SALES		OPERATIONS		
TURNER	SALES	BLAKE	SALES				OPERATIONS
MARTIN	SALES	BLAKE	SALES				
WARD	SALES	BLAKE	SALES				
ALLEN	SALES	BLAKE	SALES				
MILLER	ACCOUNTING	CLARK	ACCOUNTING				
ADAMS	RESEARCH	SCOTT	RESEARCH				
CLARK	ACCOUNTING	KING	ACCOUNTING				
BLAKE	SALES	KING	ACCOUNTING				
JONES	RESEARCH	KING	ACCOUNTING				

UT 06.02 - El Lenguaje DQL. JOINS.

5.- Uniones externas.

5.3.- Uniones por ambos lados. FULL OUTER JOIN. Ejemplo (IV).

Por tanto, si se desea realizar la consulta que muestre **todos los empleados que no compartan departamento con su superior** sería la siguiente:

```
SELECT
    SUB.ENAME SUBORD,
    DSUB.DNAME DPTOSUB,
    SUP.ENAME SUPERIOR,
    DSUP.DNAME DPTOSUP
FROM EMP SUB FULL OUTER JOIN EMP SUP ON (SUB.MGR = SUP.EMPNO)
FULL OUTER JOIN DEPT DSUB ON (SUB.DEPTNO = DSUB.DEPTNO)
FULL OUTER JOIN DEPT DSUP ON (SUP.DEPTNO = DSUP.DEPTNO)
WHERE DSUB.DNAME <> DSUP.DNAME;
```

SUBORD	DPTOSUB	SUPERIOR	DPTOSUP
BLAKE	SALES	KING	ACCOUNTING
JONES	RESEARCH	KING	ACCOUNTING

UT 06.02 - El Lenguaje DQL. JOINS.

6.- Consultas Jerárquicas.

6.1.- Definición.

- No es nada raro encontrarnos con información organizada de manera jerárquica: empleados que tienen como supervisor a otro empleado, secciones que depende a su vez de otra sección de la empresa, registros de habitantes emparentados con grados de relación, etc.
- La información almacenada de esta forma puede considerarse conceptualmente **como si estuviera organizada como árbol**. En este árbol hay nodos padre de los que cuelgan nodos hijo de una forma totalmente recursiva.
- La sintaxis de una sentencia jerárquica es la siguiente:

```
SELECT  
    campos  
FROM tabla  
START WITH [condición que define el nodo raíz]  
CONNECT BY PRIOR [condición de enlace];
```

- El significado de cada cláusula es la siguiente:
 - **CONNECT BY**. Definir cómo se relacionarán la fila actual (hijo) con la fila anterior (padre). Debe ir cualificada con un operador llamado **PRIOR** definiendo así que esa expresión se refiere a la fila del padre.
 - **START WITH**. Permite identificar que elementos son considerados raíz de la estructura.

UT 06.02 - El Lenguaje DQL. JOINS.

6.- Consultas Jerárquicas.

6.2.- Ejemplo Básico (I).

En este primer ejemplo se muestra una jerárquia con la relación EMPLEADO-SUPERVISOR.

```
SELECT empno, ename, mgr
FROM emp
START WITH ename = 'KING'
CONNECT BY PRIOR empno = mgr;
```

EMPNO	ENAME	MGR
7839	KING	
7566	JONES	7839
7788	SCOTT	7566
7876	ADAMS	7788
7902	FORD	7566
7369	SMITH	7902
7698	BLAKE	7839
7499	ALLEN	7698
7521	WARD	7698
7654	MARTIN	7698
7844	TURNER	7698
7900	JAMES	7698
7782	CLARK	7839
7934	MILLER	7782

14 rows selected.

La forma en la que ejecutaría esta consulta sería la siguiente:

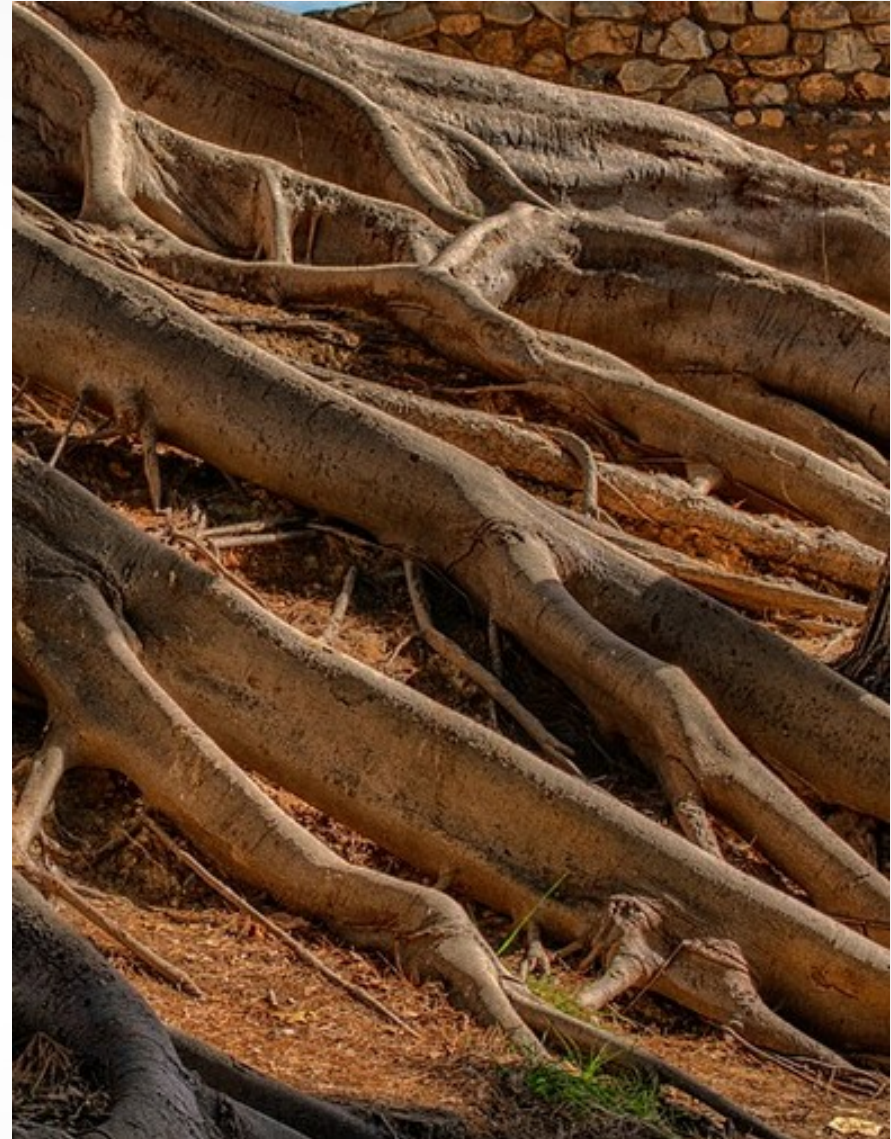
1. Selecciona las filas que son consideradas elementos raíz de la estructura evaluando la cláusula **START WITH**.

En el ejemplo sería **START WITH ename='KING'**.

2. Para cada fila considerada elemento raíz, selecciona las filas que consideradas como sus hijos. Serán aquellas que satisfagan la condición indicada en **CONNECT BY** con la fila raíz que está tratando.

En el ejemplo sería **CONNECT BY PRIOR empno = mgr**.

3. Para cada fila hija recuperada, selecciona los registros considerados como sus hijos. Es decir, que satisfacen la condición **CONNECT BY**, pero esta vez actuando la fila actual como padre y así sucesivamente.
4. Finalmente, si la consulta dispone de una cláusula **WHERE**, aplica las condiciones que no sean consideradas condiciones de **JOIN**. Los **JOIN** los realiza antes de empezar con este procesamiento.



UT 06.02 - El Lenguaje DQL. JOINS.

6.- Consultas Jerárquicas.

6.3.- Seleccionando el nodo raíz.

Podemos establecer el nodo raíz en cualquier nodo del árbol. Por ejemplo, si se desea listar el árbol desde BLAKE la sentencia es la siguiente:

```
SELECT
  ename  "EMPLEADOS"
FROM emp
START WITH ename = 'BLAKE'
CONNECT BY PRIOR empno = mgr;

EMPLEADOS
-----
BLAKE
ALLEN
WARD
MARTIN
TURNER
JAMES

6 rows selected.
```

El listado comienza directamente en BLAKE obviando los demás.

UT 06.02 - El Lenguaje DQL. JOINS.

6.- Consultas Jerárquicas.

6.4.- Eliminando nodos del arbol.

Por otro lado, podemos eliminar elementos del listado del árbol jerárquico. En ese sentido hay dos posibilidades:

- **Eliminar tan solo un nodo del árbol.** Los nodos hijos del nodo eliminado colgarán directamente del nodo padre de dicho nodo. En el ejemplo de a continuación se desea eliminar a BLAKE del listado ya que, por ejemplo este empleado ha salido de la compañía.

```
SELECT
    ename "EMPLEADO"
FROM emp
WHERE ename != 'BLAKE'
START WITH ename = 'KING'
CONNECT BY PRIOR empno = mgr;
```

Los empleados supervisados por BLAKE dependen aparecen como supervisados por el supervisor de BLAKE.

- Eliminar el nodo y todas las ramas que cuelgan de él. Para ello, **se impone la condición en la clausula CONNECT BY PRIOR.**

```
SELECT
    ename "EMPLEADO"
FROM emp
START WITH ename = 'KING'
CONNECT BY PRIOR empno = mgr AND ename != 'BLAKE';
```

UT 06.02 - El Lenguaje DQL. JOINS.

6.- Consultas Jerárquicas.

6.5.- El operador PRIOR.

- Nos podemos referir en la consulta a los campos de los nodos hijo y a los de los nodos padre.
- Para indicar que nos referimos a los campos del nodo padre anteponeamos al nombre del campo la clausula **PRIOR**.
- En este ejemplo se mostrará la frase “Empleado A es supervisado por Empleado B”.

```
SELECT
  ename || ' ES SUPERVISADO POR ' || PRIOR ename "Jerarquia"
FROM EMP
START WITH ename = 'KING'
CONNECT BY PRIOR empno = mgr;
```

Jerarquia

```
-----
KING ES SUPERVISADO POR
JONES ES SUPERVISADO POR KING
SCOTT ES SUPERVISADO POR JONES
ADAMS ES SUPERVISADO POR SCOTT
FORD ES SUPERVISADO POR JONES
SMITH ES SUPERVISADO POR FORD
BLAKE ES SUPERVISADO POR KING
ALLEN ES SUPERVISADO POR BLAKE
WARD ES SUPERVISADO POR BLAKE
MARTIN ES SUPERVISADO POR BLAKE
TURNER ES SUPERVISADO POR BLAKE
JAMES ES SUPERVISADO POR BLAKE
CLARK ES SUPERVISADO POR KING
MILLER ES SUPERVISADO POR CLARK
```

14 rows selected.

UT 06.02 - El Lenguaje DQL. JOINS.

6.- Consultas Jerárquicas.

6.6.- Nivel de un nodo desde el nodo raíz. Pseudocolumna LEVEL.

- La pseudocolumna **LEVEL** nos permite obtener el nivel de profundidad del elemento devuelto por nuestra consulta. **Nos dará la profundidad a la que está el nodo empezando por el valor uno correspondiente al nodo raíz.**

```
SELECT
  LEVEL "NIVEL",
  LPAD(ename, 2*(LEVEL - 1) + LENGTH(ename), ' ') "EMPLEADOS"
FROM emp
START WITH ename = 'KING'
CONNECT BY PRIOR empno = mgr;
```

NIVEL	EMPLEADOS
1	KING
2	JONES
3	SCOTT
4	ADAMS
3	FORD
4	SMITH
2	BLAKE
3	ALLEN
3	WARD
3	MARTIN
3	TURNER
3	JAMES
2	CLARK
3	MILLER

14 rows selected.

UT 06.02 - El Lenguaje DQL. JOINS.

6.- Consultas Jerárquicas.

6.7.- Camino desde el nodo raíz. La función SYS_CONNECT_BY_PATH.

- Esta función devuelve el camino desde la raíz al elemento devuelto. La sintaxis es la siguiente:

SYS_CONNECT_BY_PATH(campo, caracter_separador)

- Solo puede ser utilizada como dato de salida, nunca como condición en WHERE. Veamos un ejemplo:

```
SELECT
  LEVEL "NIVEL",
  ename "EMPLEADO",
  SUBSTR(SYS_CONNECT_BY_PATH(ename, ' - '), 4) "CAMINO"
FROM emp
START WITH ename = 'KING'
CONNECT BY PRIOR empno = mgr;
```

NIVEL	EMPLEADO	CAMINO
1	KING	KING
2	JONES	KING - JONES
3	SCOTT	KING - JONES - SCOTT
4	ADAMS	KING - JONES - SCOTT - ADAMS
3	FORD	KING - JONES - FORD
4	SMITH	KING - JONES - FORD - SMITH
2	BLAKE	KING - BLAKE
3	ALLEN	KING - BLAKE - ALLEN
3	WARD	KING - BLAKE - WARD
3	MARTIN	KING - BLAKE - MARTIN
3	TURNER	KING - BLAKE - TURNER
3	JAMES	KING - BLAKE - JAMES
2	CLARK	KING - CLARK
3	MILLER	KING - CLARK - MILLER

14 rows selected.

UT 06.02 - El Lenguaje DQL. JOINS.

6.- Consultas Jerárquicas.

6.8.- Nodo Intermedio vs Nodo Final. La función CONNECT_BY_ISLEAF.

- Se define los siguientes tipos de nodos:
 - **Nodo intermedio.** Aquel nodo que tiene nodos hijos colgando de él.
 - **Nodo hoja.** Aquel nodo que no tiene ningún hijo por debajo de él.

Esta pseudocolumna indica si el elemento devuelto es una hoja o no.

- En este ejemplo, la columna “STATUS” tiene como valor JEFE si tiene algún subordinado y CURRITO si no tiene ningún subordinado.:

```
SELECT
  LEVEL "NIVEL",
  ename "EMPLEADO",
  SYS_CONNECT_BY_PATH(ename, '/') "RUTA",
  DECODE(CONNECT_BY_ISLEAF, 0, 'JEFE', 'CURRITO') "STATUS"
FROM emp
START WITH ename = 'KING'
CONNECT BY PRIOR empno = mgr;
```

NIVEL	EMPLEADO	RUTA	STATUS
1	KING	/KING	JEFE
2	JONES	/KING/JONES	JEFE
3	SCOTT	/KING/JONES/SCOTT	JEFE
4	ADAMS	/KING/JONES/SCOTT/ADAMS	CURRITO
3	FORD	/KING/JONES/FORD	JEFE
4	SMITH	/KING/JONES/FORD/SMITH	CURRITO
2	BLAKE	/KING/BLAKE	JEFE
3	ALLEN	/KING/BLAKE/ALLEN	CURRITO
3	WARD	/KING/BLAKE/WARD	CURRITO
3	MARTIN	/KING/BLAKE/MARTIN	CURRITO
3	TURNER	/KING/BLAKE/TURNER	CURRITO
3	JAMES	/KING/BLAKE/JAMES	CURRITO
2	CLARK	/KING/CLARK	JEFE
3	MILLER	/KING/CLARK/MILLER	CURRITO

14 rows selected.